DISEÑO DE BASE DE DATOS

Y SCHEMA SQL

Plataforma Web de Invitaciones Digitales Personalizadas

Versión: 1.0 | Fecha: 11 de marzo de 2026

Motor: PostgreSQL 16 | ORM: TypeORM (NestJS)

Basado en: SRS v1.0 | Modelo de Negocio v1.1 | Seguridad y Datos v1.0 | API REST v1.0

Diagramas: Entidad-Relación | Clases UML | Actividad (#1, #2, #3)

Tabla de Contenidos

Tabla de Contenidos **2**
1. Introducción **3**
1.1 Propósito 3
1.2 Motor y Tecnología 3
1.3 Convenciones de Nomenclatura 3
1.4 Resumen del Modelo 3
2. Mapa de Relaciones entre Entidades **4**
3. Definición Detallada de Entidades **5**
3.1 Usuario 5
3.2 TipoEvento 5
3.3 Template 5
3.4 Servicio 6
3.5 Pedido 7
3.6 PedidoServicio 7
3.7 Invitacion 8
3.8 InvitacionServicio 9
3.9 Invitado 9
3.10 HistoriaSeccion 9
3.11 FotoAnfitrion 10
3.12 Foto 11
3.13 Musica 11
3.14 Notificacion 11
3.15 LogEliminacion 13
4. Estrategia de Índices **14**
4.1 Índices de Clave Primaria (automáticos) 14
4.2 Índices Únicos 14
4.3 Índices de Foreign Key y Consulta 14
5. Script DDL — Creación del Schema **16**
5.1 Extensiones y Configuración 16
5.2 Tablas de Catálogo y Configuración 16
5.3 Tablas de Gestión Comercial 18
5.4 Tablas de Invitaciones y Contenido 19
5.5 Tablas de Auditoría 22
6. Datos Semilla para el MVP **23**
6.1 Usuario Administrador 23
6.2 Tipos de Evento 23
6.3 Servicios 23
6.4 Templates (Placeholder MVP) 25
7. Políticas de Retención y Eliminación de Datos **26**

7.1 Retención de Datos 26
7.2 Cron Job de Eliminación Automática 26
7.3 Cascadas de Eliminación 26
7.4 Limpieza de Logs 27
8. Estrategia de Backups 28
8.1 Frecuencia y Tipo 28
8.2 Comando de Backup 28
8.3 Datos Críticos 28
9. Consultas SQL Frecuentes de Referencia 29
9.1 Obtener invitación pública con servicios 29
9.2 Conteo de confirmaciones por invitación 29
9.3 Invitaciones próximas a expirar (7 días) 29
9.4 Calcular precio dinámico de un pedido 29
9.5 Estadísticas de almacenamiento de galería 30
9.6 Dashboard de administración — Resumen 30

1. Introducción

1.1 Propósito

Este documento define el diseño completo de la base de datos para la Plataforma de Invitaciones Digitales Personalizadas. Incluye el esquema relacional con todas las entidades, tipos de datos, restricciones, índices, relaciones, scripts DDL de creación, datos semilla para el MVP, y la política de retención y eliminación de datos. El diseño está alineado con el Diagrama Entidad-Relación, el Diagrama de Clases UML y los tres Diagramas de Actividad del sistema.

1.2 Motor y Tecnología

Motor de base de datos: PostgreSQL 16+. Seleccionado por soporte nativo de UUID, JSONB, tipos de datos robustos y extensiones de seguridad.

ORM: TypeORM con NestJS + TypeScript. Todas las consultas usan parámetros preparados para prevenir inyección SQL. No se permiten strings crudos en queries.

Hosting: Hostinger VPS (mismo servidor que el backend NestJS).

Almacenamiento externo: Firebase Storage para archivos binarios (fotos, MP3). La BD solo almacena URLs de referencia.

1.3 Convenciones de Nomenclatura

Tablas: PascalCase en singular (ej: Invitacion, TipoEvento, HistoriaSeccion).

Columnas: snake_case (ej: fecha_evento, tipo_evento_id, created_at).

Claves primarias: id en todas las tablas. SERIAL para entidades estables, UUID para Invitacion.

Claves foráneas: Formato {tabla_referenciada}_id en snake_case (ej: invitacion_id, servicio_id).

Índices: Prefijo idx_{tabla}_{columna} para índices regulares, uq_{tabla}_{columna} para únicos.

Booleanos: Prefijo descriptivo (activo, habilitado, confirmado, incluido_en_base).

Timestamps: created_at con DEFAULT NOW(). Zona horaria: America/Argentina/Buenos_Aires.

1.4 Resumen del Modelo

El esquema contiene 14 tablas organizadas en 4 grupos funcionales: catálogo y configuración (Usuario, TipoEvento, Template, Servicio), gestión comercial (Pedido, PedidoServicio), invitaciones y contenido (Invitacion, InvitacionServicio, Invitado, HistoriaSeccion, FotoAnfitrion, Foto, Musica), y auditoría (Notificacion, LogEliminacion). La entidad central es Invitacion, que se relaciona directa o indirectamente con todas las demás tablas.

2. Mapa de Relaciones entre Entidades

La siguiente tabla documenta todas las relaciones del modelo, su cardinalidad y las claves foráneas involucradas. El diseño refleja fielmente el Diagrama Entidad-Relación y el Diagrama de Clases del sistema.

Origen	Card.	Destino	FK	Descripción
TipoEvento	1 : N	**Template**	`tipo_evento_id`	Un tipo de evento tiene múltiples templates
TipoEvento	1 : N	**Pedido**	`tipo_evento_id`	Un tipo categoriza múltiples pedidos
TipoEvento	1 : N	**Invitacion**	`tipo_evento_id`	Un tipo categoriza múltiples invitaciones
Template	1 : N	**Pedido**	`template_id`	Un template puede ser elegido en múltiples pedidos
Template	1 : N	**Invitacion**	`template_id`	Un template se usa en múltiples invitaciones
Pedido	1 : 1	**Invitacion**	`pedido_id`	Un pedido genera una invitación
Pedido	M : N	**Servicio**	`PedidoServicio`	Servicios seleccionados al momento del pedido
Invitacion	M : N	**Servicio**	`InvitacionServicio`	Servicios habilitados en la invitación
Invitacion	1 : N	**Invitado**	`invitacion_id`	Una invitación tiene múltiples invitados
Invitacion	1 : N (max 3)	**HistoriaSeccion**	`invitacion_id`	Hasta 3 secciones de historia
Invitacion	1 : N (max 5)	**FotoAnfitrion**	`invitacion_id`	Hasta 5 fotos del anfitrión
Invitacion	1 : N (max 1000)	**Foto**	`invitacion_id`	Galería compartida de invitados
Invitacion	1 : 0..1	**Musica**	`invitacion_id`	Máximo un archivo MP3 por invitación
Invitacion	1 : N	**Notificacion**	—	Notificaciones asociadas (referencia lógica)
Invitacion	1 : 0..1	**LogEliminacion**	`invitacion_id_ref`	Log post-eliminación (ref. blanda por UUID)

3. Definición Detallada de Entidades

Cada entidad se documenta con sus columnas, tipos de datos exactos, claves, restricciones de nulabilidad y valores por defecto. Los tipos están alineados con el Diagrama Entidad-Relación del proyecto.

3.1 Usuario

Almacena las credenciales del administrador. En el MVP existe un único usuario con rol ADMIN. Las contraseñas se almacenan con hashing Argon2/bcrypt + salting (documento de Seguridad, sección 2.2).

Campo	Tipo	PK	FK	Null	Default / Restricción
id	SERIAL	✓		NOT NULL	Auto-incremental
username	VARCHAR(100)			NOT NULL	UNIQUE
email	VARCHAR(255)			NOT NULL	
password_hash	VARCHAR(255)			NOT NULL	Argon2/bcrypt hash
role	VARCHAR(20)			NOT NULL	DEFAULT 'ADMIN'
created_at	TIMESTAMP			NOT NULL	DEFAULT NOW()

3.2 TipoEvento

Define los tipos de evento soportados. MVP incluye Bodas, Quinceañeras y Cumpleaños. Los campos específicos de cada tipo (nombres de novios, nombre de quinceañera, edad del festejado, etc.) se almacenan como JSONB para máxima flexibilidad sin alterar el esquema al agregar tipos futuros (Egresos, Comuniones, etc. en v2).

Campo	Tipo	PK	FK	Null	Default / Restricción
id	SERIAL	✓		NOT NULL	Auto-incremental
nombre	VARCHAR(100)			NOT NULL	UNIQUE
descripcion	TEXT			NULL	
campos_especificos	JSONB			NULL	Esquema dinámico
activo	BOOLEAN			NOT NULL	DEFAULT TRUE

Ejemplo JSONB campos_especificos para Bodas: { "nombres_novios": "string", "foto_pareja": "string", "ceremonia_tipo": "string", "ceremonia_hora": "time", "recepcion_hora": "time", "recepcion_lugar": "string", "trasnoche": "boolean" }

3.3 Template

Diseños visuales predefinidos por tipo de evento. El MVP requiere mínimo 3 templates por tipo (9 en total). Cada template pertenece a un único TipoEvento.

Campo	Tipo	PK	FK	Null	Default / Restricción
id	SERIAL	✓		NOT NULL	Auto-incremental
tipo_evento_id	INT		✓	NOT NULL	→ TipoEvento.id
nombre	VARCHAR(150)			NOT NULL	
thumbnail_url	VARCHAR(500)			NULL	URL de preview
descripcion	TEXT			NULL	
activo	BOOLEAN			NOT NULL	DEFAULT TRUE

3.4 Servicio

Servicios adicionales que se pueden agregar a cada invitación. Incluye servicios base (incluidos en el precio) y opcionales (con costo adicional). El precio se almacena como DECIMAL(12,2) en pesos argentinos.

Campo	Tipo	PK	FK	Null	Default / Restricción
id	SERIAL	✓		NOT NULL	Auto-incremental
nombre	VARCHAR(100)			NOT NULL	
descripcion	TEXT			NULL	
precio	DECIMAL(12,2)			NOT NULL	Precio en ARS
incluido_en_base	BOOLEAN			NOT NULL	DEFAULT FALSE
activo	BOOLEAN			NOT NULL	DEFAULT TRUE

3.5 Pedido

Registra las solicitudes que llegan desde la landing page (Diagrama de Actividad #1). El anfitrión selecciona tipo de evento, template y servicios, completa sus datos de contacto y envía el pedido. El estado evoluciona: PENDIENTE → CONTACTADO → COMPLETADO | CANCELADO, siguiendo el flujo del Diagrama de Actividad #2 donde el administrador revisa y gestiona cada pedido.

Campo	Tipo	PK	FK	Null	Default / Restricción
id	SERIAL	✓		NOT NULL	Auto-incremental
tipo_evento_id	INT		✓	NOT NULL	→ TipoEvento.id
template_id	INT		✓	NOT NULL	→ Template.id
nombre_cliente	VARCHAR(200)			NOT NULL	
telefono	VARCHAR(30)			NOT NULL	
email	VARCHAR(255)			NOT NULL	
precio_base	DECIMAL(12,2)			NOT NULL	Precio base al momento
precio_total	DECIMAL(12,2)			NOT NULL	Base + servicios
estado	VARCHAR(20)			NOT NULL	DEFAULT 'PENDIENTE'
created_at	TIMESTAMP			NOT NULL	DEFAULT NOW()

CHECK constraint: `estado IN ('PENDIENTE', 'CONTACTADO', 'COMPLETADO', 'CANCELADO')`

3.6 PedidoServicio

Tabla intermedia de la relación muchos-a-muchos entre Pedido y Servicio. Captura el precio_al_momento para mantener el historial de precios aunque el servicio cambie de precio en el futuro. Clave primaria compuesta (pedido_id, servicio_id).

Campo	Tipo	PK	FK	Null	Default / Restricción
pedido_id	INT	✓	✓	NOT NULL	→ Pedido.id (PK compuesta)
servicio_id	INT	✓	✓	NOT NULL	→ Servicio.id (PK compuesta)
precio_al_momento	DECIMAL(12,2)			NOT NULL	Precio capturado

3.7 Invitacion

Entidad central del sistema. Cada invitación es una página web personalizada para un evento. Se identifica con UUID v4 para evitar enumeración de IDs secuenciales (requisito de seguridad NFR-3.5). Contiene toda la información del evento, coordenadas para Google Maps, personalización visual y configuración de la galería. La fecha_expiracion se calcula como fecha_evento + 3 meses, momento en que el cron job nocturno ejecutará la eliminación automática (RF-5.3).

Campo	Tipo	PK	FK	Null	Default / Restricción
id	UUID	✓		NOT NULL	DEFAULT gen_random_uuid()
pedido_id	INT		✓	NULL	→ Pedido.id
template_id	INT		✓	NOT NULL	→ Template.id
tipo_evento_id	INT		✓	NOT NULL	→ TipoEvento.id
titulo	VARCHAR(200)			NOT NULL	
fecha_evento	DATE			NOT NULL	
hora_evento	TIME			NOT NULL	
ubicacion	VARCHAR(300)			NOT NULL	Nombre del lugar
direccion	VARCHAR(500)			NOT NULL	Dirección completa
latitud	DECIMAL(10,7)			NOT NULL	Para Google Maps
longitud	DECIMAL(10,7)			NOT NULL	Para Google Maps
color_primario	VARCHAR(7)			NULL	Formato #RRGGBB
contrasena_asistentes	VARCHAR(255)			NULL	Hash para lista asistentes
max_fotos	INT			NOT NULL	DEFAULT 1000
campos_especificos	JSONB			NULL	Datos dinámicos del evento
activa	BOOLEAN			NOT NULL	DEFAULT TRUE
created_at	TIMESTAMP			NOT NULL	DEFAULT NOW()
fecha_expiracion	DATE			NOT NULL	evento + 3 meses

URL pública: `invitaciones.com/{id}`

URL personalizada: `invitaciones.com/{id}?invitado=nombre-apellido`

3.8 InvitacionServicio

Tabla intermedia entre Invitacion y Servicio. Define qué servicios están habilitados para cada invitación. El flag habilitado permite activar/desactivar un servicio sin eliminarlo de la relación. Clave primaria compuesta (invitacion_id, servicio_id).

Campo	Tipo	PK	FK	Null	Default / Restricción
invitacion_id	UUID	✓	✓	NOT NULL	→ Invitacion.id (PK compuesta)
servicio_id	INT	✓	✓	NOT NULL	→ Servicio.id (PK compuesta)
habilitado	BOOLEAN			NOT NULL	DEFAULT TRUE

3.9 Invitado

Almacena los datos de cada invitado individual asociado a una invitación. Se carga masivamente desde un JSON (Diagrama de Actividad #2, paso "Carga JSON con lista de invitados"). El campo confirmado y fecha_confirmacion se actualizan cuando el invitado presiona "Confirmar Asistencia" en la página del evento (Diagrama de Actividad #3). Solo la primera confirmación cuenta (idempotente).

Campo	Tipo	PK	FK	Null	Default / Restricción
id	SERIAL	✓		NOT NULL	Auto-incremental
invitacion_id	UUID		✓	NOT NULL	→ Invitacion.id
nombre	VARCHAR(100)			NOT NULL	
apellido	VARCHAR(100)			NOT NULL	
confirmado	BOOLEAN			NOT NULL	DEFAULT FALSE
fecha_confirmacion	TIMESTAMP			NULL	Se llena al confirmar

3.10 HistoriaSeccion

Secciones de historia de la invitación. Cada invitación puede tener hasta 3 secciones, cada una con un párrafo de texto (máx. 3000 caracteres) y una imagen. El texto se sanitiza para prevenir XSS (eliminando etiquetas <script> y eventos inline). El orden determina la posición visual en la invitación.

Campo	Tipo	PK	FK	Null	Default / Restricción
id	SERIAL	✓		NOT NULL	Auto-incremental
invitacion_id	UUID		✓	NOT NULL	→ Invitacion.id
texto	VARCHAR(3000)			NOT NULL	Sanitizado HTML

imagen_url	VARCHAR(500)			NULL	URL en Firebase Storage
orden	SMALLINT			NOT NULL	1, 2 o 3

Restricción de aplicación: Máximo 3 registros por invitacion_id (validado en backend, no en BD).

3.11 FotoAnfitrion

Fotos cargadas por el administrador al crear la invitación (Diagrama de Actividad #2). Hasta 5 fotos por invitación, una de las cuales puede usarse como header. Se valida tipo MIME y tamaño en el backend antes de subir a Firebase Storage.

Campo	Tipo	PK	FK	Null	Default / Restricción
id	SERIAL	✓		NOT NULL	Auto-incremental
invitacion_id	UUID		✓	NOT NULL	→ Invitacion.id
url	VARCHAR(500)			NOT NULL	URL en Firebase Storage
orden	SMALLINT			NOT NULL	Posición de la foto
tamano	INT			NOT NULL	Tamaño en bytes

3.12 Foto

Fotos de la galería compartida, subidas por los invitados al escanear el código QR (Diagrama de Actividad #3). Máximo 1000 fotos por invitación, 15 MB por archivo. Se valida tipo MIME real del contenido (no solo extensión). Se almacenan en Firebase Storage con retención de 3 meses post-evento.

Campo	Tipo	PK	FK	Null	Default / Restricción
id	SERIAL	✓		NOT NULL	Auto-incremental
invitacion_id	UUID		✓	NOT NULL	→ Invitacion.id
url	VARCHAR(500)			NOT NULL	URL en Firebase Storage
tamano	INT			NOT NULL	Tamaño en bytes
mime_type	VARCHAR(50)			NOT NULL	ej: image/jpeg
created_at	TIMESTAMP			NOT NULL	DEFAULT NOW()

3.13 Musica

Archivo MP3 asociado a la invitación. Relación 1:0..1 con Invitacion (UNIQUE constraint en invitacion_id). Máximo 20 MB. Se reproduce automáticamente con volumen bajo tras aceptar el modal de bienvenida (Diagrama de Actividad #3). Tipo MIME verificado como audio/mpeg.

Campo	Tipo	PK	FK	Null	Default / Restricción
id	SERIAL	✓		NOT NULL	Auto-incremental
invitacion_id	UUID		✓	NOT NULL	→ Invitacion.id, UNIQUE
archivo_url	VARCHAR(500)			NOT NULL	URL en Firebase Storage
tamano	INT			NOT NULL	Tamaño en bytes
mime_type	VARCHAR(50)			NOT NULL	DEFAULT 'audio/mpeg'

3.14 Notificacion

Registra las notificaciones automáticas enviadas por email. Tipos: NUEVO_PEDIDO (al propietario cuando llega un pedido), EXPIRACION_PROXIMA (al anfitrión 7 y 3 días antes de la eliminación), AVISO_ELIMINACION (registro interno del cron job). Identificada en el Diagrama de Clases con métodos enviar(), programar(fecha) y marcarEnviada().

Campo	Tipo	PK	FK	Null	Default / Restricción
id	SERIAL	✓		NOT NULL	Auto-incremental
tipo	VARCHAR(30)			NOT NULL	Enum de tipo

destinatario_email	VARCHAR(255)			NOT NULL	
asunto	VARCHAR(255)			NOT NULL	
mensaje	TEXT			NOT NULL	
enviada	BOOLEAN			NOT NULL	DEFAULT FALSE
fecha_envio	TIMESTAMP			NULL	Se llena al enviar
created_at	TIMESTAMP			NOT NULL	DEFAULT NOW()

CHECK constraint: `tipo IN ('NUEVO_PEDIDO', 'EXPIRACION_PROXIMA', 'AVISO_ELIMINACION')`

3.15 LogEliminacion

Registro de auditoría de las eliminaciones automáticas ejecutadas por el cron job nocturno (RF-5.3). Usa invitacion_id_ref como referencia blanda (UUID) ya que el registro de la invitación fue eliminado (hard delete). Los logs se retienen por 6 meses (documento de Seguridad, sección 1.1).

Campo	Tipo	PK	FK	Null	Default / Restricción
id	SERIAL	✓		NOT NULL	Auto-incremental
invitacion_id_ref	UUID			NOT NULL	Referencia blanda (no FK)
fecha_eliminacion	TIMESTAMP			NOT NULL	DEFAULT NOW()
fotos_eliminadas	INT			NOT NULL	DEFAULT 0
detalles	TEXT			NULL	Información adicional

4. Estrategia de Índices

Los índices se diseñan para optimizar las consultas más frecuentes del sistema según los flujos de los Diagramas de Actividad. Se priorizan las búsquedas por clave foránea (joins), filtros por estado, y consultas de expiración para el cron job. Los índices UNIQUE refuerzan restricciones de negocio a nivel de BD.

4.1 Índices de Clave Primaria (automáticos)

PostgreSQL crea automáticamente un índice B-tree sobre cada columna definida como PRIMARY KEY. No es necesario crearlos manualmente.

4.2 Índices Únicos

Nombre del Índice	Columna(s)	Tipo	Justificación
`uq_usuario_username`	`username`	UNIQUE	Login del admin (no duplicados)
`uq_tipo_evento_nombre`	`nombre`	UNIQUE	Tipos de evento sin duplicados
`uq_musica_invitacion`	`invitacion_id`	UNIQUE	Máximo 1 MP3 por invitación
`uq_invitado_inv_nombre`	`invitacion_id, nombre, apellido`	UNIQUE	Sin invitados duplicados por invitación

4.3 Índices de Foreign Key y Consulta

Nombre del Índice	Columna(s)	Tipo	Justificación
`idx_template_tipo_evento`	`tipo_evento_id`	B-tree	Filtrar templates por tipo de evento
`idx_pedido_estado`	`estado`	B-tree	Listar pedidos por estado en admin
`idx_pedido_created`	`created_at`	B-tree	Ordenar pedidos por fecha
`idx_invitacion_pedido`	`pedido_id`	B-tree	Buscar invitación por pedido
`idx_invitacion_activa`	`activa`	B-tree parcial	WHERE activa = TRUE
`idx_invitacion_expiracion`	`fecha_expiracion`	B-tree	Cron job de eliminación automática
`idx_invitacion_fecha_evento`	`fecha_evento`	B-tree	Reportes de próximos eventos
`idx_invitado_invitacion`	`invitacion_id`	B-tree	Listar invitados de una invitación
`idx_invitado_confirmado`	`invitacion_id, confirmado`	B-tree	Conteo de confirmaciones
`idx_foto_invitacion`	`invitacion_id`	B-tree	Listar fotos de galería
`idx_foto_created`	`created_at`	B-tree	Ordenar fotos por fecha de subida
`idx_foto_anfitrion_inv`	`invitacion_id`	B-tree	Fotos del anfitrión por invitación

`idx_historia_invitacion`	`invitacion_id`	B-tree	Secciones de historia por invitación
`idx_notificacion_tipo`	`tipo`	B-tree	Filtrar notificaciones por tipo
`idx_notificacion_enviada`	`enviada`	B-tree parcial	WHERE enviada = FALSE (pendientes)
`idx_log_elim_fecha`	`fecha_eliminacion`	B-tree	Auditoría y limpieza de logs

5. Script DDL — Creación del Schema

A continuación se presenta el script SQL completo para crear todas las tablas, restricciones, índices y extensiones necesarias. El script es idempotente (usa IF NOT EXISTS donde aplica) y está ordenado por dependencias para evitar errores de referencia.

5.1 Extensiones y Configuración

```
-- Habilitar extensión para UUID v4
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Zona horaria por defecto
SET timezone = 'America/Argentina/Buenos_Aires';
```

5.2 Tablas de Catálogo y Configuración

```
-- ═══════════════════════════════════════════
-- TABLA: Usuario
-- ═══════════════════════════════════════════
CREATE TABLE Usuario (
  id            SERIAL        PRIMARY KEY,
  username      VARCHAR(100)  NOT NULL UNIQUE,
  email         VARCHAR(255)  NOT NULL,
  password_hash VARCHAR(255)  NOT NULL,
  role          VARCHAR(20)   NOT NULL DEFAULT 'ADMIN',
  created_at    TIMESTAMP     NOT NULL DEFAULT NOW()
);

-- ═══════════════════════════════════════════
-- TABLA: TipoEvento
-- ═══════════════════════════════════════════
CREATE TABLE TipoEvento (
  id                  SERIAL        PRIMARY KEY,
  nombre              VARCHAR(100)  NOT NULL UNIQUE,
  descripcion         TEXT,
  campos_especificos  JSONB,
  activo              BOOLEAN       NOT NULL DEFAULT TRUE
);

-- ═══════════════════════════════════════════
-- TABLA: Template
-- ═══════════════════════════════════════════
CREATE TABLE Template (
  id              SERIAL        PRIMARY KEY,
  tipo_evento_id  INT           NOT NULL
    REFERENCES TipoEvento(id),
  nombre          VARCHAR(150)  NOT NULL,
  thumbnail_url   VARCHAR(500),
  descripcion     TEXT,
  activo          BOOLEAN       NOT NULL DEFAULT TRUE
);

CREATE INDEX idx_template_tipo_evento
  ON Template(tipo_evento_id);

-- ═══════════════════════════════════════════
-- TABLA: Servicio
```

```
-- ==========================================
CREATE TABLE Servicio (
  id               SERIAL         PRIMARY KEY,
  nombre           VARCHAR(100)   NOT NULL,
  descripcion      TEXT,
  precio           DECIMAL(12,2)  NOT NULL,
  incluido_en_base BOOLEAN        NOT NULL DEFAULT FALSE,
  activo           BOOLEAN        NOT NULL DEFAULT TRUE
);
```

5.3 Tablas de Gestión Comercial

```
-- ═══════════════════════════════════════════
-- TABLA: Pedido
-- ═══════════════════════════════════════════
CREATE TABLE Pedido (
  id              SERIAL         PRIMARY KEY,
  tipo_evento_id  INT            NOT NULL
    REFERENCES TipoEvento(id),
  template_id     INT            NOT NULL
    REFERENCES Template(id),
  nombre_cliente  VARCHAR(200)   NOT NULL,
  telefono        VARCHAR(30)    NOT NULL,
  email           VARCHAR(255)   NOT NULL,
  precio_base     DECIMAL(12,2)  NOT NULL,
  precio_total    DECIMAL(12,2)  NOT NULL,
  estado          VARCHAR(20)    NOT NULL
    DEFAULT 'PENDIENTE'
    CHECK (estado IN (
      'PENDIENTE','CONTACTADO',
      'COMPLETADO','CANCELADO'
    )),
  created_at      TIMESTAMP      NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_pedido_estado
  ON Pedido(estado);
CREATE INDEX idx_pedido_created
  ON Pedido(created_at);

-- ═══════════════════════════════════════════
-- TABLA: PedidoServicio (M:N intermedia)
-- ═══════════════════════════════════════════
CREATE TABLE PedidoServicio (
  pedido_id         INT            NOT NULL
    REFERENCES Pedido(id) ON DELETE CASCADE,
  servicio_id       INT            NOT NULL
    REFERENCES Servicio(id),
  precio_al_momento DECIMAL(12,2)  NOT NULL,
  PRIMARY KEY (pedido_id, servicio_id)
);
```

5.4 Tablas de Invitaciones y Contenido

```
-- ═══════════════════════════════════════════
-- TABLA: Invitacion (CENTRAL)
-- ═══════════════════════════════════════════
CREATE TABLE Invitacion (
  id                      UUID           PRIMARY KEY
    DEFAULT gen_random_uuid(),
  pedido_id               INT
    REFERENCES Pedido(id),
  template_id             INT            NOT NULL
    REFERENCES Template(id),
  tipo_evento_id          INT            NOT NULL
    REFERENCES TipoEvento(id),
  titulo                  VARCHAR(200)   NOT NULL,
  fecha_evento            DATE           NOT NULL,
  hora_evento             TIME           NOT NULL,
  ubicacion               VARCHAR(300)   NOT NULL,
  direccion               VARCHAR(500)   NOT NULL,
  latitud                 DECIMAL(10,7)  NOT NULL,
  longitud                DECIMAL(10,7)  NOT NULL,
  color_primario          VARCHAR(7),
  contrasena_asistentes   VARCHAR(255),
  max_fotos               INT            NOT NULL DEFAULT 1000,
  campos_especificos      JSONB,
  activa                  BOOLEAN        NOT NULL DEFAULT TRUE,
  created_at              TIMESTAMP      NOT NULL DEFAULT NOW(),
  fecha_expiracion        DATE           NOT NULL
);

CREATE INDEX idx_invitacion_pedido
  ON Invitacion(pedido_id);
CREATE INDEX idx_invitacion_activa
  ON Invitacion(activa) WHERE activa = TRUE;
CREATE INDEX idx_invitacion_expiracion
  ON Invitacion(fecha_expiracion);
CREATE INDEX idx_invitacion_fecha_evento
  ON Invitacion(fecha_evento);

-- ═══════════════════════════════════════════
-- TABLA: InvitacionServicio (M:N intermedia)
-- ═══════════════════════════════════════════
CREATE TABLE InvitacionServicio (
  invitacion_id  UUID     NOT NULL
    REFERENCES Invitacion(id) ON DELETE CASCADE,
  servicio_id    INT      NOT NULL
    REFERENCES Servicio(id),
  habilitado     BOOLEAN  NOT NULL DEFAULT TRUE,
  PRIMARY KEY (invitacion_id, servicio_id)
);

-- ═══════════════════════════════════════════
-- TABLA: Invitado
-- ═══════════════════════════════════════════
CREATE TABLE Invitado (
  id                  SERIAL       PRIMARY KEY,
  invitacion_id       UUID         NOT NULL
    REFERENCES Invitacion(id) ON DELETE CASCADE,
  nombre              VARCHAR(100) NOT NULL,
  apellido            VARCHAR(100) NOT NULL,
  confirmado          BOOLEAN      NOT NULL DEFAULT FALSE,
  fecha_confirmacion  TIMESTAMP,
  UNIQUE (invitacion_id, nombre, apellido)
);
```

```
CREATE INDEX idx_invitado_invitacion
  ON Invitado(invitacion_id);
CREATE INDEX idx_invitado_confirmado
  ON Invitado(invitacion_id, confirmado);
```

```
-- ═══════════════════════════════════════════
-- TABLA: HistoriaSeccion
-- ═══════════════════════════════════════════
CREATE TABLE HistoriaSeccion (
  id              SERIAL         PRIMARY KEY,
  invitacion_id   UUID           NOT NULL
    REFERENCES Invitacion(id) ON DELETE CASCADE,
  texto           VARCHAR(3000)  NOT NULL,
  imagen_url      VARCHAR(500),
  orden           SMALLINT       NOT NULL
    CHECK (orden BETWEEN 1 AND 3)
);

CREATE INDEX idx_historia_invitacion
  ON HistoriaSeccion(invitacion_id);

-- ═══════════════════════════════════════════
-- TABLA: FotoAnfitrion
-- ═══════════════════════════════════════════
CREATE TABLE FotoAnfitrion (
  id              SERIAL        PRIMARY KEY,
  invitacion_id   UUID          NOT NULL
    REFERENCES Invitacion(id) ON DELETE CASCADE,
  url             VARCHAR(500)  NOT NULL,
  orden           SMALLINT      NOT NULL,
  tamano          INT           NOT NULL
);

CREATE INDEX idx_foto_anfitrion_inv
  ON FotoAnfitrion(invitacion_id);

-- ═══════════════════════════════════════════
-- TABLA: Foto (Galería de invitados)
-- ═══════════════════════════════════════════
CREATE TABLE Foto (
  id              SERIAL        PRIMARY KEY,
  invitacion_id   UUID          NOT NULL
    REFERENCES Invitacion(id) ON DELETE CASCADE,
  url             VARCHAR(500)  NOT NULL,
  tamano          INT           NOT NULL,
  mime_type       VARCHAR(50)   NOT NULL,
  created_at      TIMESTAMP     NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_foto_invitacion
  ON Foto(invitacion_id);
CREATE INDEX idx_foto_created
  ON Foto(created_at);

-- ═══════════════════════════════════════════
-- TABLA: Musica
-- ═══════════════════════════════════════════
CREATE TABLE Musica (
  id              SERIAL        PRIMARY KEY,
  invitacion_id   UUID          NOT NULL UNIQUE
    REFERENCES Invitacion(id) ON DELETE CASCADE,
  archivo_url     VARCHAR(500)  NOT NULL,
  tamano          INT           NOT NULL,
  mime_type       VARCHAR(50)   NOT NULL
    DEFAULT 'audio/mpeg'
);
```

5.5 Tablas de Auditoría

```
-- ═══════════════════════════════════════════
-- TABLA: Notificacion
-- ═══════════════════════════════════════════
CREATE TABLE Notificacion (
  id                  SERIAL        PRIMARY KEY,
  tipo                VARCHAR(30)   NOT NULL
    CHECK (tipo IN (
      'NUEVO_PEDIDO',
      'EXPIRACION_PROXIMA',
      'AVISO_ELIMINACION'
    )),
  destinatario_email  VARCHAR(255)  NOT NULL,
  asunto              VARCHAR(255)  NOT NULL,
  mensaje             TEXT          NOT NULL,
  enviada             BOOLEAN       NOT NULL DEFAULT FALSE,
  fecha_envio         TIMESTAMP,
  created_at          TIMESTAMP     NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_notificacion_tipo
  ON Notificacion(tipo);
CREATE INDEX idx_notificacion_enviada
  ON Notificacion(enviada) WHERE enviada = FALSE;

-- ═══════════════════════════════════════════
-- TABLA: LogEliminacion
-- ═══════════════════════════════════════════
CREATE TABLE LogEliminacion (
  id                  SERIAL     PRIMARY KEY,
  invitacion_id_ref   UUID       NOT NULL,
  fecha_eliminacion   TIMESTAMP  NOT NULL DEFAULT NOW(),
  fotos_eliminadas    INT        NOT NULL DEFAULT 0,
  detalles            TEXT
);

CREATE INDEX idx_log_elim_fecha
  ON LogEliminacion(fecha_eliminacion);
```

6. Datos Semilla para el MVP

Script de inserción inicial con los datos mínimos necesarios para el funcionamiento del MVP. Incluye el usuario administrador, los 3 tipos de evento, los servicios definidos en el Modelo de Negocio y templates placeholder.

6.1 Usuario Administrador

```
-- El hash debe generarse en la aplicación (Argon2/bcrypt)
-- Este es un placeholder; reemplazar con hash real
INSERT INTO Usuario (username, email, password_hash, role)
VALUES (
  'admin',
  'admin@invitaciones.com',
  '$argon2id$v=19$m=65536,t=3,p=4$PLACEHOLDER',
  'ADMIN'
);
```

6.2 Tipos de Evento

```
INSERT INTO TipoEvento (nombre, descripcion, campos_especificos) VALUES
('Boda', 'Celebración de matrimonio', '{
  "nombres_novios": "string",
  "foto_pareja": "string",
  "ceremonia_tipo": "string",
  "ceremonia_hora": "time",
  "recepcion_hora": "time",
  "recepcion_lugar": "string",
  "trasnoche": "boolean"
}'::jsonb),

('Quinceañera', 'Fiesta de 15 años', '{
  "nombre_quinceañera": "string",
  "foto_quinceañera": "string",
  "vals_pareja": "string",
  "vals_cancion": "string",
  "tematica": "string",
  "dress_code": "string"
}'::jsonb),

('Cumpleaños', 'Fiesta de cumpleaños', '{
  "nombre_festejado": "string",
  "foto_festejado": "string",
  "edad": "number",
  "tipo_fiesta": "string",
  "actividades": "string"
}'::jsonb);
```

6.3 Servicios

```
INSERT INTO Servicio
  (nombre, descripcion, precio, incluido_en_base)
VALUES
  ('Información del evento',
```

```
 'Fecha, hora, ubicación, dress code, campos editables',
 0.00, TRUE),

('Mapa con ubicación',
 'Google Maps embebido + link copeable + abrir nativo',
 0.00, TRUE),

('Fotos del anfitrión',
 'Hasta 5 fotos del anfitrión en la invitación',
 0.00, TRUE),

('Confirmación de Asistencia',
 'Botón RSVP + panel confirmados + exportar CSV',
 10000.00, FALSE),

('Cuenta Regresiva',
 'Countdown dinámico días/h/m/s hasta el evento',
 10000.00, FALSE),

('Música',
 'MP3 con controles. Activa al aceptar modal',
 10000.00, FALSE),

('Historia',
 'Sección texto + fotos (máx 3000 chars, 3 secciones)',
 10000.00, FALSE),

('Galería de Fotos (QR)',
 'QR para subir fotos, máx 1000, 15 MB c/u, ZIP',
 50000.00, FALSE),

('Múltiples Tarjetas',
 'Segunda versión de la invitación (50% desc.)',
 15000.00, FALSE);
```

6.4 Templates (Placeholder MVP)

```
-- 3 templates por tipo de evento = 9 templates MVP
-- tipo_evento_id: 1=Boda, 2=Quinceañera, 3=Cumpleaños

INSERT INTO Template
  (tipo_evento_id, nombre, descripcion) VALUES
  -- Bodas
  (1, 'Boda Clásica', 'Diseño elegante en tonos dorados'),
  (1, 'Boda Moderna', 'Diseño minimalista contemporáneo'),
  (1, 'Boda Rústica', 'Diseño campestre con texturas naturales'),
  -- Quinceañeras
  (2, 'Quinceañera Princesa', 'Diseño con coronas y detalles reales'),
  (2, 'Quinceañera Moderna', 'Diseño juvenil con colores vibrantes'),
  (2, 'Quinceañera Elegante', 'Diseño sofisticado en tonos pastel'),
  -- Cumpleaños
  (3, 'Cumpleaños Festivo', 'Diseño colorido con globos y confetti'),
  (3, 'Cumpleaños Elegante', 'Diseño adulto sofisticado'),
  (3, 'Cumpleaños Infantil', 'Diseño divertido para niños');
```

7. Políticas de Retención y Eliminación de Datos

7.1 Retención de Datos

Las políticas de retención se definen en el documento de Seguridad y Datos (sección 1.1) y están alineadas con la Ley 25.326 de Protección de Datos Personales de Argentina.

Dato	Retención	Acción al vencer
Invitación completa	3 meses post-evento	Hard delete de BD + Firebase Storage
Galería de fotos	3 meses post-evento	Hard delete de Firebase Storage
Archivo MP3	3 meses post-evento	Hard delete de Firebase Storage
Datos de invitados	3 meses post-evento	Hard delete (CASCADE desde Invitacion)
Secciones de historia	3 meses post-evento	Hard delete (CASCADE desde Invitacion)
Logs de eliminación	6 meses	Hard delete por cron job separado
Datos de pedidos	Indefinido	Se conservan como historial comercial
Notificaciones	Indefinido	Se conservan como auditoría

7.2 Cron Job de Eliminación Automática

Se ejecuta cada noche a las 02:00 AM (America/Argentina/Buenos_Aires). Identifica invitaciones cuyo campo fecha_expiracion es anterior a la fecha actual. Para cada invitación expirada ejecuta en orden: registro de LogEliminacion con conteo de fotos y detalles, eliminación de archivos en Firebase Storage (galería + MP3 + fotos anfitrión), eliminación del registro en tabla Invitacion (las tablas hijas se eliminan por CASCADE: Invitado, HistoriaSeccion, FotoAnfitrion, Foto, Musica, InvitacionServicio).

Query de identificación:

```
SELECT id, titulo, fecha_evento, fecha_expiracion
FROM Invitacion
WHERE fecha_expiracion < CURRENT_DATE
  AND activa = TRUE;
```

7.3 Cascadas de Eliminación

Todas las tablas hijas de Invitacion tienen ON DELETE CASCADE en sus claves foráneas. Esto garantiza que al eliminar una invitación, todos los datos asociados se eliminan automáticamente de la BD sin queries adicionales. Los archivos en Firebase Storage deben eliminarse explícitamente en el código de la aplicación antes del DELETE en BD.

Tablas con CASCADE desde Invitacion: InvitacionServicio, Invitado, HistoriaSeccion, FotoAnfitrion, Foto, Musica.

Tablas con CASCADE desde Pedido: PedidoServicio.

7.4 Limpieza de Logs

Un segundo cron job mensual elimina registros de LogEliminacion cuyo campo fecha_eliminacion tiene más de 6 meses de antigüedad, cumpliendo con la política de retención de logs definida en el documento de Seguridad.

```
DELETE FROM LogEliminacion
WHERE fecha_eliminacion < CURRENT_DATE - INTERVAL '6 months';
```

8. Estrategia de Backups

8.1 Frecuencia y Tipo

Backups automáticos semanales de la base de datos PostgreSQL completa (pg_dump). Los datos clasificados como críticos (confirmaciones de asistencia) no deben perderse bajo ninguna circunstancia (NFR-2.3). Las fotos de galería e información de invitaciones también se respaldan semanalmente.

8.2 Comando de Backup

```
# Backup completo semanal (cron: domingos 03:00 AM)
pg_dump -U postgres -Fc invitaciones_db \
  > /backups/invitaciones_$(date +%Y%m%d).dump

# Restauración
pg_restore -U postgres -d invitaciones_db \
  /backups/invitaciones_20260315.dump
```

8.3 Datos Críticos

Las confirmaciones de asistencia se clasifican como datos críticos que no pueden perderse. Si ocurre una falla entre backups, las confirmaciones son irrecuperables. Se recomienda implementar WAL (Write-Ahead Logging) con archivado continuo para minimizar la ventana de pérdida de datos.

9. Consultas SQL Frecuentes de Referencia

Consultas optimizadas para los flujos más comunes del sistema, alineadas con los Diagramas de Actividad.

9.1 Obtener invitación pública con servicios

```
SELECT i.*, t.nombre AS template_nombre,
       te.nombre AS tipo_evento_nombre,
       json_agg(DISTINCT jsonb_build_object(
         'id', s.id, 'nombre', s.nombre
       )) FILTER (WHERE ise.habilitado = TRUE)
         AS servicios_habilitados
FROM Invitacion i
JOIN Template t ON i.template_id = t.id
JOIN TipoEvento te ON i.tipo_evento_id = te.id
LEFT JOIN InvitacionServicio ise
  ON i.id = ise.invitacion_id
LEFT JOIN Servicio s ON ise.servicio_id = s.id
WHERE i.id = $1 AND i.activa = TRUE
GROUP BY i.id, t.nombre, te.nombre;
```

9.2 Conteo de confirmaciones por invitación

```
SELECT
  COUNT(*) AS total_invitados,
  COUNT(*) FILTER (WHERE confirmado = TRUE)
    AS total_confirmados,
  COUNT(*) FILTER (WHERE confirmado = FALSE)
    AS total_pendientes
FROM Invitado
WHERE invitacion_id = $1;
```

9.3 Invitaciones próximas a expirar (7 días)

```
SELECT id, titulo, fecha_evento, fecha_expiracion,
  (fecha_expiracion - CURRENT_DATE) AS dias_restantes
FROM Invitacion
WHERE fecha_expiracion BETWEEN CURRENT_DATE
  AND CURRENT_DATE + INTERVAL '7 days'
  AND activa = TRUE
ORDER BY fecha_expiracion ASC;
```

9.4 Calcular precio dinámico de un pedido

```
SELECT
  30000.00 AS precio_base,
  COALESCE(SUM(s.precio), 0) AS servicios_total,
  30000.00 + COALESCE(SUM(s.precio), 0)
    AS precio_total
FROM Servicio s
```

```
WHERE s.id = ANY($1::int[])  -- Array de IDs
  AND s.activo = TRUE
  AND s.incluido_en_base = FALSE;
```

9.5 Estadísticas de almacenamiento de galería

```
SELECT
  i.id, i.titulo, i.max_fotos,
  COUNT(f.id) AS fotos_actuales,
  COALESCE(SUM(f.tamano), 0) AS bytes_usados,
  ROUND(COALESCE(SUM(f.tamano), 0)
    / 1048576.0, 2) AS mb_usados
FROM Invitacion i
LEFT JOIN Foto f ON i.id = f.invitacion_id
WHERE i.id = $1
GROUP BY i.id;
```

9.6 Dashboard de administración — Resumen

```
SELECT
  (SELECT COUNT(*) FROM Pedido
    WHERE estado = 'PENDIENTE') AS pedidos_pendientes,
  (SELECT COUNT(*) FROM Invitacion
    WHERE activa = TRUE) AS invitaciones_activas,
  (SELECT COUNT(*) FROM Invitacion
    WHERE fecha_expiracion
      BETWEEN CURRENT_DATE
      AND CURRENT_DATE + INTERVAL '7 days'
  ) AS proximas_expirar,
  (SELECT COUNT(*) FROM Invitacion
    WHERE fecha_expiracion < CURRENT_DATE
      AND activa = TRUE
  ) AS pendientes_eliminacion;
```